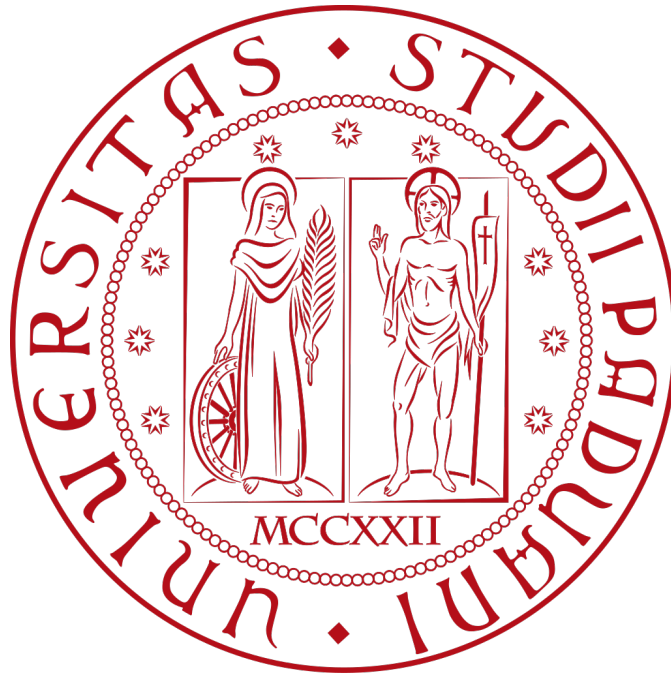




ZUCCHETTI



AI TrAIning

Piano di progetto

Filippo Sabbadin

08 / 08 / 2025

Indice

1. Introduzione	1
1.1. Scopo del documento	1
1.2. Scopo del prodotto	1
1.3. Glossario	2
1.4. Riferimenti	2
1.4.1. Riferimenti normativi	2
1.4.2. Riferimenti informativi	2
1.4.3. Riferimenti tecnici	2
2. Tecnologie	3
3. Architettura	4
3.1. Introduzione	4
3.1.1. Concetti chiave di Godot	4
3.1.2. Assenza di interfacce	4
3.2. Assets comuni	4
3.2.1. Giocatore	5
3.2.1.1. Telecamera	6
3.2.1.2. State Machine	7
3.2.1.3. Collezionabili	8
3.2.2. Interazione	8
3.2.2.1. Cartelli	9
3.2.2.2. NPC	9
3.2.2.3. Dialoghi	9
3.2.3. Salvataggi	10
3.2.4. LevelLoader	10
3.3. Menu	10
3.3.1. Menu di pausa	10
3.3.2. Menu principale	10
3.4. Struttura base livello	10
3.5. Livello «Regressione lineare»	10
3.5.1. Cannone	10
3.5.2. Grafico	10
3.6. Livello «Albero di decisione»	10
3.6.1. Albero	10
3.7. Livello «Causalità»	10
3.7.1. Scena di intermezzo	11
4. Requisiti soddisfatti	12
4.1. Tabella requisiti soddisfatti	12

4.2. Grafico requisiti soddisfatti	12
--	----

Lista di immagini

Figura 1	Diagramma delle classi del giocatore	5
Figura 2	Diagramma delle classi della telecamera del giocatore	6
Figura 3	Diagramma sulla struttura della macchina di stati	7
Figura 4	Diagramma degli oggetti con cui il giocatore può interagire	8
Figura 5	Diagramma sul funzionamento di un dialogo	9
Figura 6	Diagramma del livello della causalità	11

Lista di tabelle

Tabella 1 Tecnologie utilizzate	3
--	----------

1. Introduzione

1.1. Scopo del documento

Lo scopo di questo documento è descrivere l'architettura e le scelte relative a essa che sono state fatte durante la fase di progettazione e codifica del progetto.

Vengono riportati i grafici UML delle classi per rappresentare l'architettura finale dell'applicazione.

1.2. Scopo del prodotto

Il progetto consiste nello sviluppo di un videogioco con il motore di gioco [Godot*](#) di tipo [platformer*](#), in cui il giocatore controlla un personaggio che si muove in un ambiente tridimensionale. Il gioco si basa su temi di [Intelligenza Artificiale*](#) e [Machine Learning*](#), sviluppando meccaniche ed elementi del livello basati su questi argomenti.

Comprende 3 livelli, ognuno con un tema diverso, un menu principale che viene visualizzato non appena il gioco viene avviato, e un menu di pausa che può essere visualizzato in qualsiasi momento durante il gioco.

Ogni livello presenta meccaniche diverse e uniche.

- **Regressione lineare**

Il livello presenta una serie di grafici con una linea e dei dati, accedendo ad un «cannone» il giocatore può posizionare dei nuovi dati nel grafico e la retta si modifica in base alla posizione del nuovo punto. Nel caso i punti siano stati piazzati male e non sia possibile modificare ulteriormente la linea, il giocatore può resettare il grafico premendo il tasto apposito.

Se soddisfatto della rotazione e posizione della linea il giocatore può uscire dal cannone e camminare sopra la linea per proseguire nel livello.

- **Albero di decisione**

In questo livello sono presenti diversi cani, ognuno di una razza diversa, ed un albero di decisione tridimensionale. Ad ogni nodo dell'albero viene mostrata una domanda e la direzione da seguire per ogni risposta.

L'obiettivo del giocatore è rispondere correttamente a ognuna di queste domande e posizionare il cane nel nodo finale giusto. In caso corretto, il giocatore potrà visualizzare la razza indovinata in un tabellone, e questo darà un Training Data al giocatore per ogni razza indovinata.

Infine è presente un NPC che permette al giocatore di riportare tutti i cani nella posizione

iniziale nel caso siano troppo sparsi.

- **Causalità**

Appena caricato nel livello, il giocatore avrà la possibilità di parlare con un NPC gelataio che lo guiderà all'obiettivo di questo livello, cioè accendere tutte le unità di codizionatori esterne presenti, poiché crede che le vendite di gelati e l'uso dei condizionatori siano correlate. Nel livello sono presenti anche dei grafici che cambiano durante il livello in base alle azioni del giocatore.

Una volta accese tutte le unità, verrà avviata una [scena di intermezzo*](#) e alla fine di essa il giocatore dovrà indicare la vera causa della vendita di gelati.

1.3. Glossario

1.4. Riferimenti

1.4.1. Riferimenti normativi

1.4.2. Riferimenti informativi

1.4.3. Riferimenti tecnici

- Documentazione di Godot:

<https://docs.godotengine.org/it/4.x/about/introduction.html>

- I concetti chiave di Godot:

https://docs.godotengine.org/it/4.x/getting_started/introduction/key_concepts_overview.html

- La filosofia progettuale di Godot:

https://docs.godotengine.org/it/4.x/getting_started/introduction/godot_design_philosophy.html

2. Tecnologie

Nome	Descrizione	Versione
Codice		
GDScript		(Legata a Godot)
Python		
Typst	Linguaggio utilizzato per la stesura dei documenti	
*.csv	«Comma seprated values», file utilizzato per memorizzare le frasi nelle lingue diverse supportate dal gioco	-
*.ini	Tipo d file «plain-text» utilizzato per salvare i dati del gioco	-
Softwares		
Godot	Il motore di gioco open source per lo sviluppo del videogioco.	4.5-beta3-mono
Blender	Software di modellazione ed animazione 3D usato per creare i modelli 3D del gioco	4.4.3
GIMP	Software di modifica di immagini, usato per modificare le textures del gioco	3.0.4
Strumenti e servizi		
Git		2.50.1
GitHub Actions		-

Tabella 1: Tecnologie utilizzate

3. Architettura

3.1. Introduzione

3.1.1. Concetti chiave di Godot

Nodo
Scena
Segnali

3.1.2. Assenza di interfacce

3.2. Assets comuni

Di seguito viene mostrata l'architettura degli [assets](#)* presenti in più parti del gioco che non sono unici ad un livello specifico.

3.2.1. Giocatore

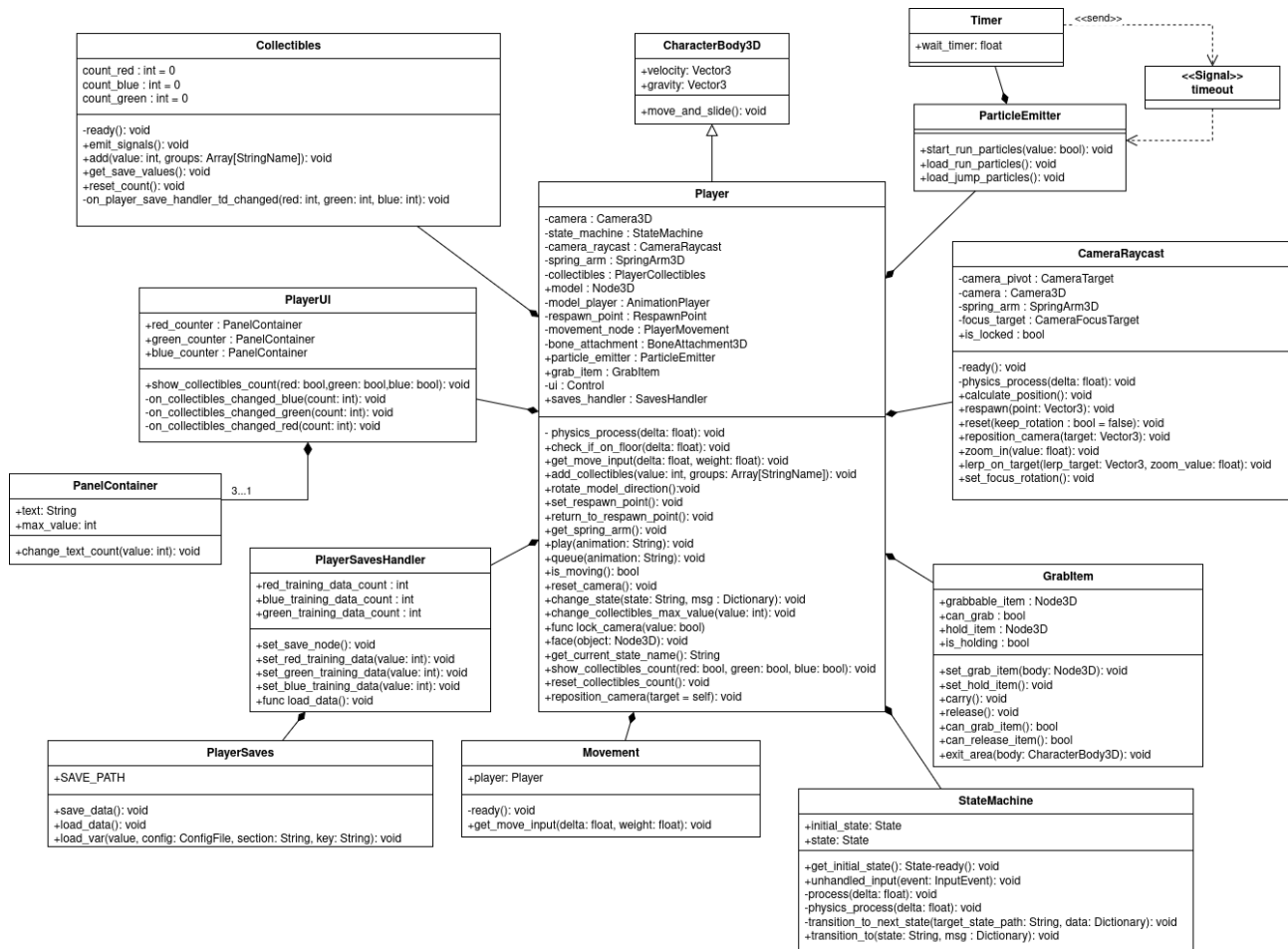


Figura 1: Diagramma delle classi del giocatore

Il giocatore può essere considerata la classe principale di tutta l'applicazione, attraverso il quale l'utente può interagire con la maggior parte dell'applicazione. Nonostante ci sia solo un giocatore presente del gioco, questo non è un [singleton*](#), visto che non è presente nel menu principale all'avvio del gioco.

Molte variabili presenti nel giocatore sono references ai suoi nodi figli presenti nella scena, queste variabili sono precedute dalla parola chiave *@onready* nel codice. Similmente, molte funzioni della classe servono solo per accedere alle variabili degli oggetti aggregati.

L'architettura del giocatore presenta diverse funzionalità:

- **telecamera;**
- **macchina di stati;**
- **raccolta di collezionabili;**
- **gestione dei salvataggi;**

- raccolta di oggetti;
- emissione di particelle 3D;
- UI.

3.2.1.1. Telecamera

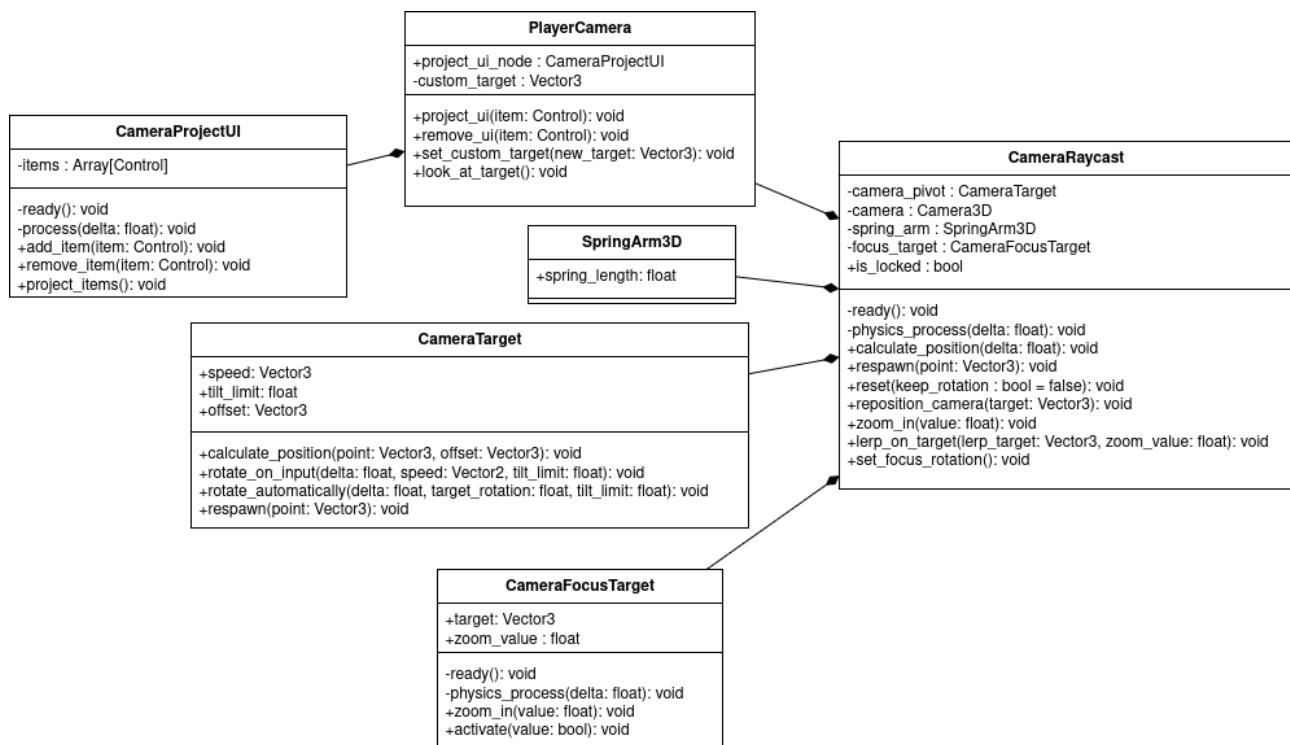


Figura 2: Diagramma delle classi della telecamera del giocatore

3.2.1.2. State Machine

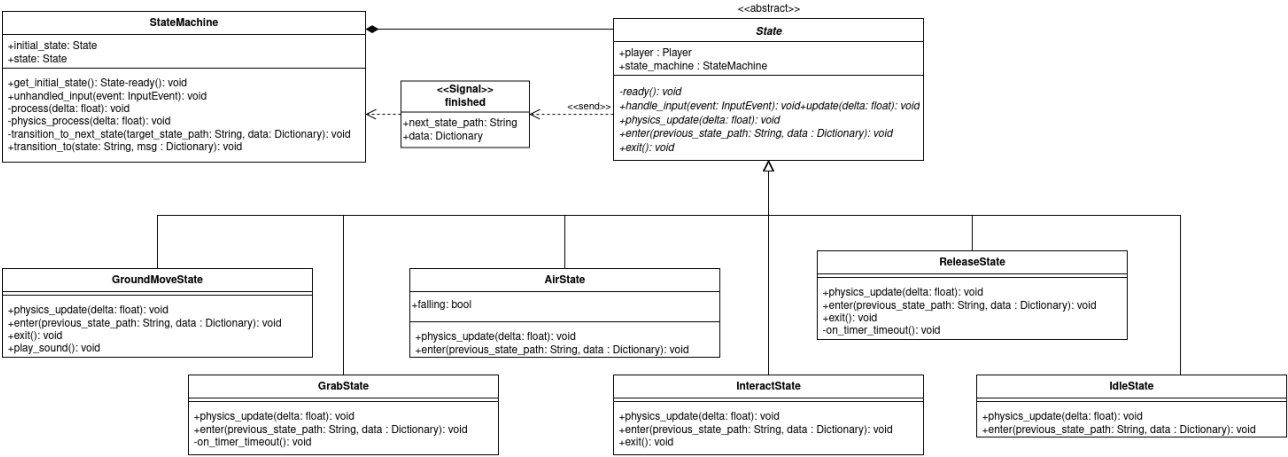


Figura 3: Diagramma sulla struttura della macchina di stati

3.2.1.3. Collezionabili

3.2.2. Interazione

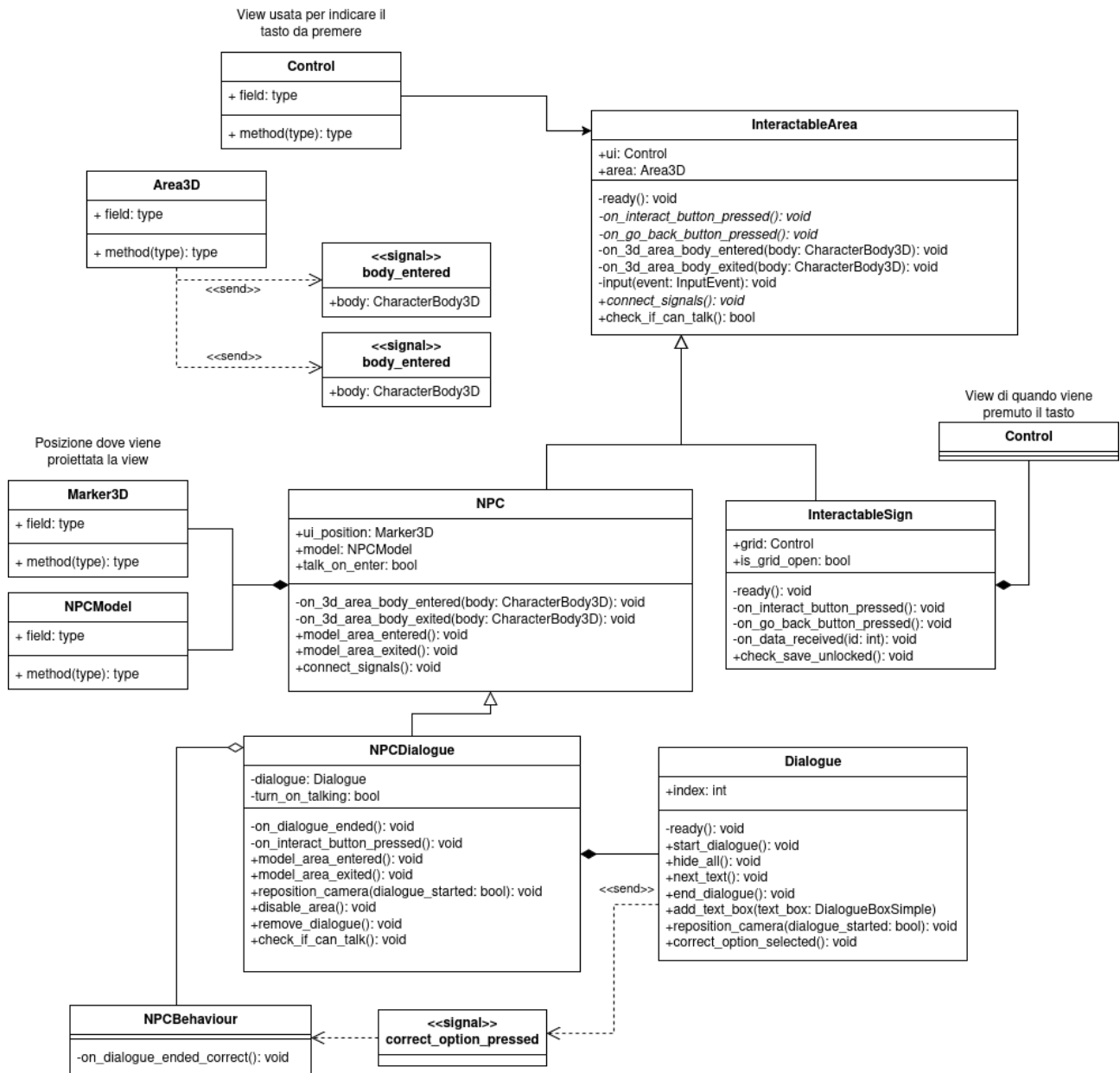


Figura 4: Diagramma degli oggetti con cui il giocatore può interagire

3.2.2.1. Cartelli

3.2.2.2. NPC

3.2.2.3. Dialoghi

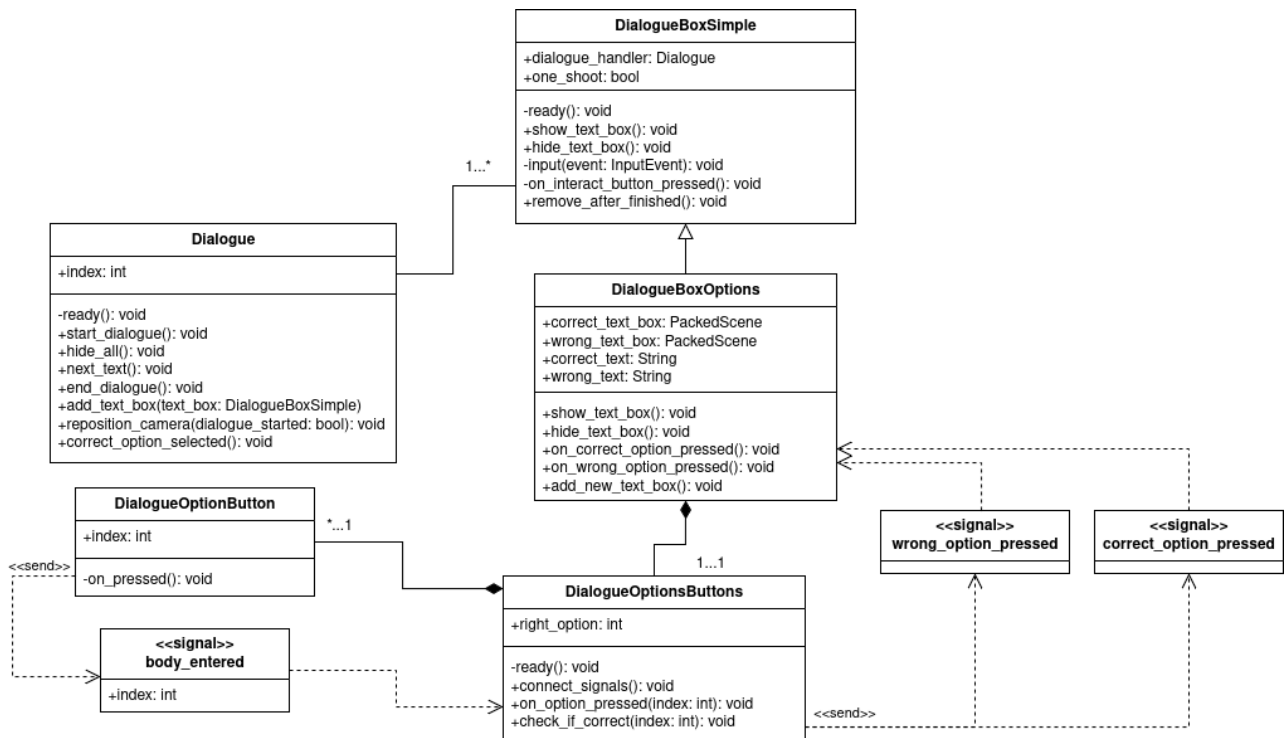


Figura 5: Diagramma sul funzionamento di un dialogo

3.2.3. Salvataggi

3.2.4. LevelLoader

3.3. Menu

3.3.1. Menu di pausa

3.3.2. Menu principale

3.4. Struttura base livello

3.5. Livello «Regressione lineare»

3.5.1. Cannone

3.5.2. Grafico

3.6. Livello «Albero di decisione»

3.6.1. Albero

3.7. Livello «Causalità»

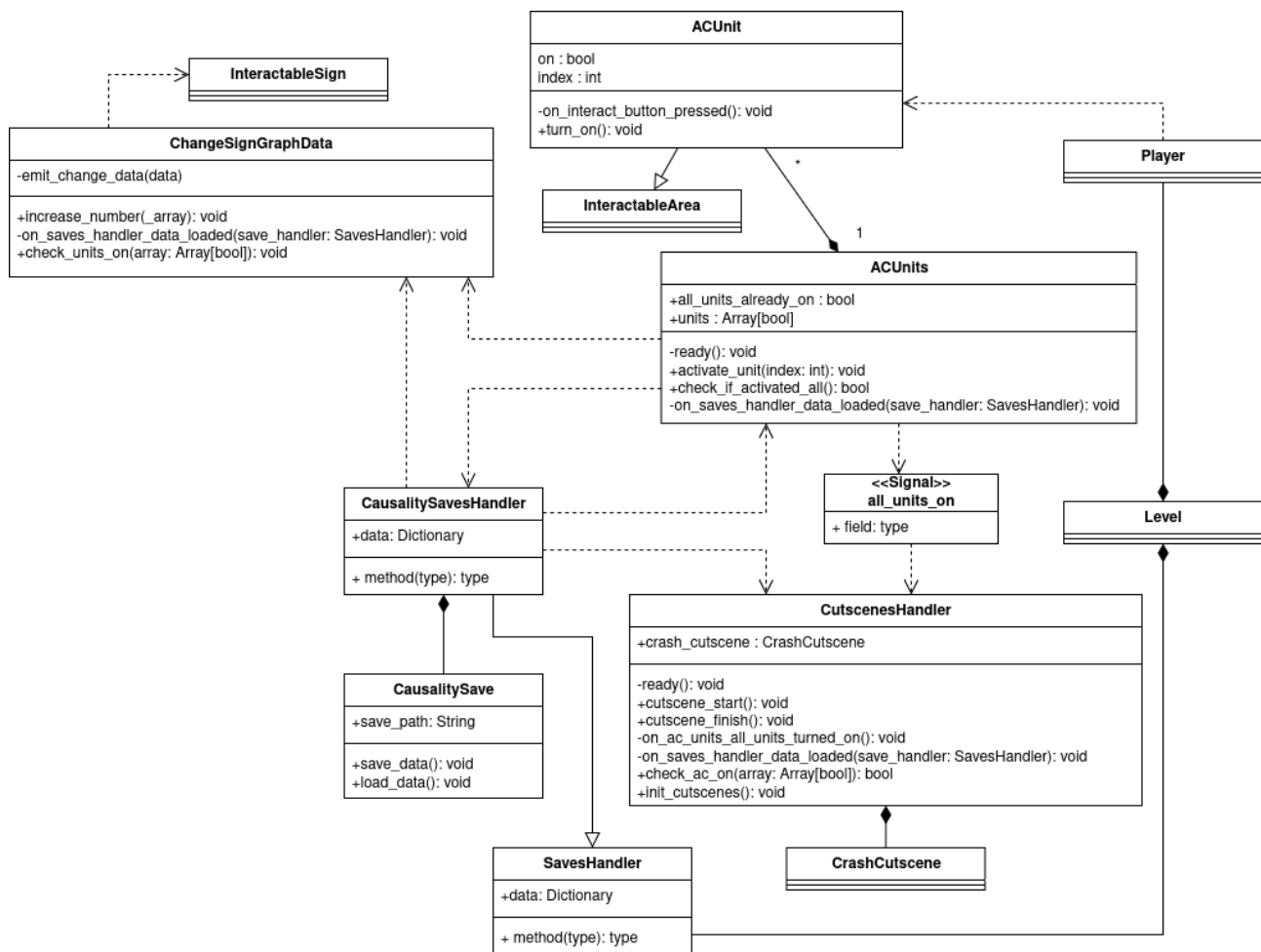


Figura 6: Diagramma del livello della causalità

3.7.1. Scena di intermezzo

4. Requisiti soddisfatti

4.1. Tabella requisiti soddisfatti

4.2. Grafico requisiti soddisfatti